

NGINX JS DOM — Full-Proxy Programming Guide

Contents

1. Introduction	2
1.1. What JS-Pilgrim Provides	2
1.2. The Layered Architecture	3
1.3. How to Read This Guide	4
2. Request Phase Hooks	4
2.1. Overview	4
2.2. Location Hooks: <code>location.addHook(fn)</code>	4
2.3. Server Hooks: <code>server.addHook(fn)</code>	5
2.4. Global HTTP Hooks: <code>nginx.http.addHook(fn)</code>	5
2.5. Execution Order	5
2.6. Async Hooks	6
2.7. Middleware Chains: <code>next()</code>	6
2.8. Per-Request Context: <code>req.ctx</code>	7
3. Response Hooks	7
3.1. <code>location.addResponseHook(fn)</code>	7
4. Upstream Filters	8
4.1. Response Body: <code>location.addUpstreamFilter(fn)</code>	8
4.2. Request Body: <code>location.addUpstreamRequestFilter(fn)</code>	8
4.3. Chaining	9
5. Accept Hooks	9
5.1. On JS-Managed Listeners: <code>listener.on('accept', fn)</code>	9
5.2. On Standard Listen Sockets: <code>server.on('accept', fn)</code>	9
5.3. <code>NginxConnection</code>	10
5.4. Constraints	10
6. L4 Stream Filters	10
6.1. Inbound: <code>addL4Filter(fn)</code>	10
6.1.1. On JS-managed listeners	10
6.1.2. On standard listen sockets	11
6.1.3. Generator protocol	11
6.1.4. Filter chaining	11
6.2. Outbound: <code>addL4SendFilter(fn)</code>	12
6.2.1. On JS-managed listeners	12
6.2.2. On standard listen sockets	12
6.3. Availability Matrix	12

6.4. Async vs. Sync Inside the Generator	12
7. Cross-Worker Shared State	13
7.1. <code>nginx.shared</code>	13
7.2. Method Reference	13
7.3. Limits	14
7.4. When <code>nginx.shared</code> Is Available	14
7.5. Concurrency	14
7.6. Persistence Across Hot-Reload	14
8. Master Lifecycle Events	14
8.1. <code>nginx.on(event, fn)</code>	14
8.2. Event Reference	15
8.3. Constraints	15
9. Module System	15
9.1. Loading Filesystem Plugins: <code>nginx.use(path[, config])</code>	15
9.1.1. Entry-point resolution	16
9.1.2. <code>package.json</code> format	16
9.1.3. Inside the plugin script	16
9.2. Installing Inline Plugins: <code>nginx.install(plugin[, config])</code>	16
9.3. Hot Reload: <code>nginx.use()</code> from a Request Handler	17
10. Complete Examples	17
10.1. Authentication Middleware	17
10.2. Rate Limiting at the TCP Level	18
10.3. Middleware Timing Wrapper	18
10.4. L4 Protocol Negotiation	19
10.5. Cross-Worker Hit Counter Plugin	19
11. Reference	20
11.1. Hook API Summary	20
11.2. Accept Hook API Summary	20
11.3. L4 Filter API Summary	20
11.4. Upstream Filter API Summary	21
11.5. <code>nginx.on</code> Event Reference	21
11.6. <code>nginx.shared</code> Limits	21
11.7. Error Handling Summary	21

1. Introduction

1.1. What JS-Pilgrim Provides

The NGINX JS DOM exposes the nginx configuration tree as a JavaScript object model (COM) and lets JavaScript handle HTTP requests. **JS-Pilgrim** is the layer above the COM: it gives JavaScript complete control over every phase of connection and request processing.

Without JS-Pilgrim, JavaScript can read and mutate configuration, assign request handlers to locations, and install body filters (see the [Filters Guide](#)). With JS-Pilgrim, JavaScript also controls:

- **Accept hooks** — fire immediately after `accept(2)`, before HTTP parsing. Inspect or reject connections by IP address, rate limit at TCP level.
- **L4 stream filters** — transform raw byte streams in both directions before or after HTTP processing, without writing a C module.
- **Request phase hooks** — run before the content handler at location, server, or global HTTP scope. Async hooks, middleware-style `next()` wrapping, and early `req.respond()` short-circuit are all supported.
- **Response hooks** — modify response headers after the content handler has selected them, before they are sent to the client.
- **Upstream filters** — transform the body sent to or received from a proxied backend, in addition to the client-side body filters.
- **Per-request context** — a plain JS object (`req.ctx`) that persists across async suspend/resume and across the access-phase → content-phase boundary.
- **Cross-worker shared state** — `nginx.shared`, a flat key/value store backed by shared memory, visible to all worker processes simultaneously.
- **Master lifecycle events** — `nginx.on('workerSpawned', fn)` and friends let JS react to nginx process events.
- **Module packaging** — `nginx.use(path, config)` loads directory-based plugin packages; `nginx.install(fn, config)` installs inline plugins.

1.2. The Layered Architecture

Every incoming connection passes through a stack of independently programmable layers:

Client TCP		
□		
[Accept hooks]	listener.on / server.on	(sync)
□		
[L4 inbound filters]	listener/server.addL4Filter	(async gen*)
□		
[HTTP parsing]	nginx built-in	
□		
[Access phase hooks]	nginx.http.addHook / server.addHook	(async, next())
□		
[Content phase hooks]	location.addHook	(async, next())
□		
[Content handler]	location.handler or nginx built-in	
□		
[Header filter]	location.addResponseHook	(sync)
□		
[Body filter]	location.addBodyFilter	(see Filters Guide)
□		

```

[Upstream req filter] location.addUpstreamRequestFilter      (async gen*)
[]
[Upstream response filter] location.addUpstreamFilter        (async gen*)
[]
[L4 outbound filters] listener/server.addL4SendFilter        (async gen*)
[]
Client TCP

```

A layer that does nothing passes through automatically. A layer that calls `req.respond()` (or `conn.reject()`) short-circuits everything below it.

1.3. How to Read This Guide

This guide covers JS-Pilgrim hooks and filters. For the COM object tree (`nginx.http.servers[]`, upstream config, SSL settings, etc.) see the [Developer Manual](#). For body filters (`addBodyFilter`, `addHeaderFilter`) see the [Filters Guide](#). For dynamic reconfiguration (`write_mode`, `snapshot()`, `restore()`) see the [Reconfiguration Guide](#).

All JS-Pilgrim hooks must be registered from a `js_source` script that runs at config time (before workers are forked), unless stated otherwise.

2. Request Phase Hooks

2.1. Overview

Request phase hooks are JS callbacks that run inside nginx's access or content phase, before the content handler produces a response. They share a three- outcome model:

Outcome	Behaviour
Return normally (no <code>req.respond()</code>)	Hook passes — the chain advances to the next hook or to the content handler.
<code>req.respond(status, headers, body)</code>	Chain stops — remaining hooks and the content handler are skipped.
<code>throw</code> / rejected Promise	nginx logs the error and sends a 500 response.

2.2. Location Hooks: `location.addHook(fn)`

Location hooks fire in the **content phase** for every request matched to that location.

```

const loc = nginx.http.servers[0].locations[0];

// Synchronous hook
loc.addHook(function(req) {
  if (!req.headers['x-api-key']) {
    req.respond(401, {}, 'Missing API key\n');
  }
});

```

```

    }
  });

  // Asynchronous hook
  loc.addHook(async function(req) {
    const result = await nginx.setTimeout(10); // yield to event loop
    req.respond(200, {}, 'ok\n');
  });

```

fn

`function(req[, next])` or `async function(req[, next])`. `req` is a `NginxRequest`. See [Middleware Chains](#): `next()` for the optional `next` argument.

Returns `location` for chaining. Multiple `addHook` calls register multiple hooks; they run in registration order.

2.3. Server Hooks: `server.addHook(fn)`

Server hooks fire in the **access phase** for every request that matches the virtual server, regardless of which location ultimately handles the request.

```

nginx.http.servers[0].addHook(async function(req) {
  if (req.uri.startsWith('/internal/')) {
    req.respond(403, {}, 'Forbidden\n');
  }
});

```

2.4. Global HTTP Hooks: `nginx.http.addHook(fn)`

Global HTTP hooks fire in the **access phase** for every request handled by the HTTP block, across all virtual servers.

```

nginx.http.addHook(function(req) {
  nginx.log(nginx.INFO,
    '[global] ' + req.method + ' ' + req.uri);
});

```

2.5. Execution Order

For a given request, hooks execute in the order: global HTTP hooks → server hooks → content-phase (location) hooks → content handler.

Within each scope, hooks run in the order they were registered.

2.6. Async Hooks

Any hook callback may be an **async function**. When a hook returns a pending Promise, nginx suspends the request (incrementing `r→main→count`) and returns `NGX_DONE`. The event loop continues processing other connections. When the Promise settles, `ngx_js_async_check` resumes the request.

```
nginx.http.addHook(async function(req) {
  // Simulate an async token validation call
  await nginx.setTimeout(20);

  var token = req.headers['authorization'] || '';
  if (!token.startsWith('Bearer ')) {
    req.respond(401, {}, 'Unauthorized\n');
  }
  // otherwise: pass through to next hook / content handler
});
```

Only one request is suspended per worker at a time (serialised through the event loop). For high-concurrency async gating, use a SharedWorker.

2.7. Middleware Chains: `next()`

Every hook callback receives an optional second argument `next`. Calling `next()` runs the remaining hooks in the chain and returns a Promise that resolves when they complete. Code after `await next()` runs before the content handler.

```
// Wrap an entire hook chain with timing
loc.addHook(async function(req, next) {
  var t0 = Date.now();
  await next();           // run remaining hooks
  var elapsed = Date.now() - t0;
  nginx.log(nginx.INFO, 'hook chain took ' + elapsed + 'ms');
  // content handler runs after this function returns
});

loc.addHook(function(req) {
  req.respond(200, {}, 'hello\n');
});
```

Hooks that do not call `next()` auto-advance (backward-compatible with pre-P9 code). Calling `next()` more than once returns the cached result.

The chain runner is a Koa-style middleware stack:

```
hook A (calls next())
  hook B (calls next())
    [no more hooks → next() resolves immediately]
```

```
hook B post-next code
hook A post-next code
[chain settles → content handler runs]
```

2.8. Per-Request Context: `req.ctx`

`req.ctx` is a plain JS object allocated on first access and tied to the nginx request pool. It persists across async suspend/resume and across the access-phase → content-phase boundary, making it the right place to pass data from an access hook to the content handler.

```
// Access-phase hook: validate and store decoded token
nginx.http.addHook(async function(req) {
  var token = parseJWT(req.headers['authorization']);
  if (!token) {
    req.respond(401, {}, 'Unauthorized\n');
    return;
  }
  req.ctx.user = token.sub;    // survives async suspend
  req.ctx.role = token.role;
});

// Content handler: read token data stored by the hook
loc.handler = function(req) {
  req.respond(200, {}, 'Hello, ' + req.ctx.user + '\n');
};
```

`req.ctx` is lazily created on first access. It is freed automatically via `ngx_pool_cleanup_add` when the request ends. It is `undefined` before first access (reading it does not allocate).

3. Response Hooks

3.1. `location.addResponseHook(fn)`

Response hooks fire after the content handler has set response headers, before they are sent to the client.

```
loc.addResponseHook(function(req) {
  // Inspect and mutate response headers
  req.setResponseHeader('x-powered-by', 'nginx-js');
  req.removeResponseHeader('server');

  // Read or change the status code
  if (req.status === 404) {
    req.status = 200;
    // body is already determined at this point
  }
});
```

```
});
```

fn

function(req) — synchronous only. The return value is ignored. Async callbacks are accepted but the headers will have been sent before any awaited value resolves.

Returns **location** for chaining. Multiple calls register multiple response hooks; they run in registration order.

4. Upstream Filters

Upstream filters run in the body-filter chain but only when the location proxies to an upstream (**proxy_pass**, **fastcgi_pass**, etc.). They are independent of **addBodyFilter**.

4.1. Response Body: **location.addUpstreamFilter(fn)**

Transforms the upstream's response body before it reaches the client.

```
loc.addUpstreamFilter(async function*(body, req) {
    // body – full upstream response body as a string
    yield body.replace(/internal-hostname/g, 'public-hostname');
});

// Chain two upstream response filters
loc.addUpstreamFilter(async function*(body, req) {
    yield body.toUpperCase();
});
loc.addUpstreamFilter(async function*(body, req) {
    yield '[' + body + ']'; // receives output of first filter
});
```

fn

async function*(body, req). **body** is the full buffered response body string. Yield zero or more string chunks; they are concatenated to form the final output.

4.2. Request Body: **location.addUpstreamRequestFilter(fn)**

Transforms the client's request body before it is forwarded to the upstream.

```
loc.addUpstreamRequestFilter(async function*(body, req) {
    // body – full client request body as a string
    var obj = JSON.parse(body);
    obj.processed_by = 'nginx-js';
    yield JSON.stringify(obj);
});
```

```
});
```

fn

Same signature as `addUpstreamFilter`. `body` is the complete (buffered) client request body. The transformed output replaces `r→request_body→bufs` and updates `Content-Length` before the proxy module runs.

4.3. Chaining

Multiple `addUpstreamFilter` calls chain in registration order: the output of filter N becomes the input of filter N+1. The same applies to `addUpstreamRequestFilter`.

5. Accept Hooks

Accept hooks fire immediately after `accept(2)`, before nginx begins HTTP parsing. They can inspect or reject connections based on TCP-level information.

5.1. On JS-Managed Listeners: `listener.on('accept', fn)`

For listeners created with `nginx.http.attach(sock)`:

```
const sock = nginx.createSocket('0.0.0.0:8443');
const lst = nginx.http.attach(sock);
lst.addServer(defaultSrv);

lst.on('accept', function(conn) {
  if (!isAllowed(conn.remoteAddr)) {
    conn.reject();
  }
});
```

5.2. On Standard Listen Sockets: `server.on('accept', fn)`

For sockets declared with nginx's `listen` directive:

```
// server {} whose listen directive owns port 8080
var srv = nginx.http.servers.find(function(s) {
  return s.name === 'my-server';
});

srv.on('accept', function(conn) {
  nginx.log(nginx.INFO,
    'new connection from ' + conn.remoteAddr);
});
```

```
});
```

`server.on()` installs the hook during `init_conf` by replacing `ls→handler` on every listen socket whose default server is `srv`.

5.3. NginxConnection

Both APIs deliver a `NginxConnection` object to the callback:

Property / Method	Type	Description
<code>conn.remoteAddr</code>	string (read-only)	Client IP address, e.g. "192.168.1.10". IPv4 and IPv6.
<code>conn.remotePort</code>	number (read-only)	Client TCP port, e.g. 54321.
<code>conn.reject()</code>	void	Close the connection immediately. All remaining accept hooks are skipped; nginx never starts HTTP processing for this connection.

5.4. Constraints

Accept hooks are **synchronous only**. Returning a pending Promise from an accept hook causes the connection to be rejected and a warning to be logged.

Multiple `on('accept', fn)` calls register multiple hooks; they run in registration order. If any hook calls `conn.reject()`, subsequent hooks are not called.

6. L4 Stream Filters

L4 filters intercept raw byte streams at the TCP level, before or after HTTP processing. They are expressed as **async generator functions** — the JS-Pilgrim stream primitive.

6.1. Inbound: `addL4Filter(fn)`

Intercepts bytes travelling from client to nginx, before the HTTP parser sees them.

6.1.1. On JS-managed listeners

```
listener.addL4Filter(async function*(source) {
  for await (const chunk of source) {
    // chunk – Uint8Array of raw TCP bytes
    yield chunk; // pass through unchanged
    return;
  }
});
```

6.1.2. On standard listen sockets

```
srv.addL4Filter(async function*(source) {
  for await (const chunk of source) {
    const s = String.fromCharCode(...chunk);
    yield s.startsWith('MAGIC:') ? s.slice(6) : s;
    return;
  }
});
```

6.1.3. Generator protocol

The **source** argument is a [async iterable](#):

```
for await (const chunk of source) {
  // chunk: Uint8Array – one TCP segment
  // await is permitted here
  yield transform(chunk); // send transformed bytes downstream
  // return to stop after first chunk (common for prefix-stripping)
}
```

Yield value	Meaning
Uint8Array	Raw bytes copied verbatim.
string	Bytes interpreted as UTF-8 / ASCII characters.
No yield before return	No bytes emitted (HTTP parser receives empty input — likely a 400 response).
throw / exception	Connection closed; error logged.

6.1.4. Filter chaining

Multiple **addL4Filter** calls chain in registration order. The accumulated output of filter N is delivered as a single chunk to filter N+1's **source**.

```
// Filter 1: strip a 4-byte binary frame header
srv.addL4Filter(async function*(source) {
  for await (const chunk of source) {
    yield chunk.slice(4);
    return;
  }
});

// Filter 2: log the stripped payload
srv.addL4Filter(async function*(source) {
  for await (const chunk of source) {
    nginx.log(nginx.DEBUG,
```

```

        'inbound ' + chunk.length + ' bytes after strip');
    yield chunk;
    return;
  }
});

```

6.2. Outbound: `addL4SendFilter(fn)`

Intercepts bytes travelling from nginx to the client, after the HTTP response is fully formed.

6.2.1. On JS-managed listeners

```

listener.addL4SendFilter(async function*(source) {
  for await (const chunk of source) {
    yield appendSignature(chunk);
  }
});

```

6.2.2. On standard listen sockets

```

srv.addL4SendFilter(async function*(source) {
  for await (const chunk of source) {
    yield chunk; // pass through; can transform instead
  }
});

```

The generator protocol is identical to `addL4Filter`. The filter hooks into `c→send` / `c→send_chain` via pool-cleanup type-tagged state. EOF is delivered lazily inside the collect loop after the first yield.

6.3. Availability Matrix

Target	<code>addL4Filter</code>	<code>addL4SendFilter</code>
JS-managed listener (<code>nginx.http.attach(sock)</code>)	Yes	Yes
Standard <code>listen</code> socket (via <code>server.addL4Filter</code>)	Yes	Yes

6.4. Async vs. Sync Inside the Generator

`await` is fully supported inside L4 generator bodies. When the generator suspends at an `await`, the connection is parked in `w→l4_pending` and the event loop continues processing other requests. When the awaited Promise settles, `ngx_js_l4_async_check` resumes the generator.

```

// Example: async lookup before forwarding
srv.addL4Filter(async function*(source) {

```

```

    for await (const chunk of source) {
      await nginx.setTimeout(5); // simulate async work
      yield chunk;
      return;
    }
  });

```

7. Cross-Worker Shared State

7.1. `nginx.shared`

`nginx.shared` is a flat key/value store backed by a dedicated shared-memory zone. All nginx worker processes read and write the same memory region — a value written in one worker is immediately visible to every other worker.

```

// Write from any request handler
nginx.shared.set('rate:192.168.1.1', '42');

// Read from any worker
var hits = nginx.shared.get('rate:192.168.1.1'); // → '42' or undefined

// Atomic counter
nginx.shared.incr('total_requests');           // → 1, 2, 3, ...
nginx.shared.incr('total_requests', 5);         // add 5

// Cleanup
nginx.shared.delete('rate:192.168.1.1');       // → true | false
var keys = nginx.shared.keys();                 // → string[]

```

7.2. Method Reference

Method	Returns	Behaviour
<code>get(key)</code>	<code>string</code> <code>undefined</code>	Returns the stored value, or <code>undefined</code> if the key does not exist.
<code>set(key, val)</code>	<code>undefined</code>	Stores <code>val</code> for <code>key</code> . Creates or overwrites. Throws <code>RangeError</code> if key > 127 chars or value > 511 chars. Throws <code>InternalError</code> if the store is full (256 entries).
<code>delete(key)</code>	<code>boolean</code>	Removes <code>key</code> . Returns <code>true</code> if the key existed.
<code>keys()</code>	<code>string[]</code>	Returns an unsorted array of all live keys.
<code>incr(key[, delta=1])</code>	<code>number</code>	Parses the current value as a base-10 integer, adds <code>delta</code> (default 1), stores and returns the result. Non-numeric values are treated as 0; missing keys start at 0. The entire operation is atomic.

7.3. Limits

Limit	Value
Maximum key length	127 characters
Maximum value length	511 characters
Maximum entries	256
Total zone size	≈ 163 kB

7.4. When `nginx.shared` Is Available

The shared-memory zone is mapped after config parsing. `nginx.shared` methods are safe to call from request-phase handlers, hooks, and timer callbacks. Calling them from top-level `js_source` code (i.e., during `init_conf`) throws `InternalError`.

7.5. Concurrency

All five methods acquire a spinlock for the duration of the operation. `incr` is atomic — no application-level locking is required.

7.6. Persistence Across Hot-Reload

`nginx.shared` data survives `nginx -s reload`. The init callback detects the hot-reload case (`data != NULL`) and preserves existing entries.

8. Master Lifecycle Events

8.1. `nginx.on(event, fn)`

Registers a callback to be invoked in the master process when nginx lifecycle events occur.

```
nginx.on('workerSpawned', function(pid, slot) {
  nginx.log(nginx.INFO, 'worker ' + pid + ' started (slot ' + slot + ')');
});

nginx.on('workerExited', function(pid, slot, status) {
  nginx.log(nginx.WARN, 'worker ' + pid + ' exited with status ' + status);
});

nginx.on('quit',    function() { /* SIGQUIT received */ });
nginx.on('reload',  function() { /* SIGHUP received  */ });
nginx.on('reloaded', function() { /* new config active */ });
nginx.on('reopen',  function() { /* SIGUSR1 received */ });
```

Multiple calls register multiple listeners for the same event; all run in registration order.

8.2. Event Reference

Event	Callback signature	When it fires
<code>workerSpawned</code>	<code>function(pid, slot)</code>	A new worker process has been forked. <code>pid</code> — OS process ID. <code>slot</code> — index in <code>ngx_processes[]</code> .
<code>workerExited</code>	<code>function(pid, slot, status)</code>	A worker process has exited. <code>status</code> — the raw <code>waitpid</code> status value (use <code>WEXITSTATUS(status)</code> etc. in C; in JS this is the raw integer).
<code>quit</code>	<code>function()</code>	SIGQUIT received. The master is beginning graceful shutdown.
<code>reload</code>	<code>function()</code>	SIGHUP received. nginx is about to re-read the configuration file.
<code>reloaded</code>	<code>function()</code>	Hot-reload is complete and the new worker generation is active.
<code>reopen</code>	<code>function()</code>	SIGUSR1 received. Log files are being reopened (log rotation).

8.3. Constraints

- Callbacks run in the **master process** JS context, not in a worker.
- Callbacks must be synchronous. Returning a Promise is ignored.
- `nginx.on()` must be called from `js_source` (init time), not from a request handler.

9. Module System

9.1. Loading Filesystem Plugins: `nginx.use(path[, config])`

Loads a plugin package from the local filesystem.

```
nginx.use('/opt/plugins/auth-guard', { secret: process.env.AUTH_SECRET });
nginx.use('/opt/plugins/rate-limiter', { rps: 500 });
```

`path`

Absolute path, or relative to `nginx.cycle.prefix`.

`config`

Optional value exposed as `nginx.pluginConfig` before the plugin's entry script is evaluated. Defaults to `{}`.

Returns `nginx` for chaining.

9.1.1. Entry-point resolution

1. Read `<path>/package.json` if it exists.
2. If `package.json` has a `ngxjs.main` key, use `<path>/<ngxjs.main>`.
3. Otherwise fall back to `<path>/index.js`.

9.1.2. `package.json` format

Only the `ngxjs` key is read; all other fields are ignored:

```
{
  "name": "my-plugin",
  "version": "1.0.0",
  "ngxjs": {
    "main": "src/plugin.js"
  }
}
```

9.1.3. Inside the plugin script

The entry file is evaluated as an ES module. `nginx.pluginConfig` holds the `config` argument:

```
// opt/plugins/auth-guard/index.js
var cfg = nginx.pluginConfig;

nginx.http.addHook(function(req) {
  if (req.headers['x-token'] !== cfg.secret) {
    req.respond(401, {}, 'Unauthorized\n');
  }
});
```

9.2. Installing Inline Plugins: `nginx.install(plugin[, config])`

Installs a plugin without touching the filesystem.

```
// Object with .install method
nginx.install({
  install: function(cfg) {
    nginx.http.addHook(function(req) {
      req.respond(200, {}, cfg.greeting + '\n');
    });
  }
}, { greeting: 'Hello!' });
```

```
// Bare function
nginx.install(function(cfg) {
  nginx.log(nginx.INFO, 'plugin config: ' + JSON.stringify(cfg));
}, { debug: true });
```

plugin

An object with an `.install(config)` method, or a bare function called as `plugin(config)`.

Returns the return value of the plugin call.

9.3. Hot Reload: `nginx.use()` from a Request Handler

`nginx.use()` can also be called from a worker request handler (not only at init time). When called from a worker, the plugin is loaded locally, then a `NGX_CMD_JS_USE_PLUGIN` message is sent to the master, which broadcasts a `NGX_CMD_JS_LOAD_PLUGIN` command to all other workers. All workers end up with the same plugin loaded without nginx restart or reload.

```
loc.handler = async function(req) {
  var path = req.headers['x-plugin-path'];
  if (path) {
    nginx.use(path); // loads locally + broadcasts to all workers
    req.respond(200, {}, 'plugin loaded\n');
    return;
  }
  req.respond(404, {}, 'not found\n');
};
```



Hot-loaded plugins are not persisted across nginx restart. Add the `nginx.use()` call to the `js_source` script to make a plugin permanent.

10. Complete Examples

10.1. Authentication Middleware

Global auth check with per-request context passing:

```
// Global HTTP hook: validate JWT and store claims in req.ctx
nginx.http.addHook(async function(req) {
  var authHeader = req.headers['authorization'] || '';
  if (!authHeader.startsWith('Bearer ')) {
    req.respond(401, {'WWW-Authenticate': 'Bearer realm="api"'}, '');
    return;
  }
  var token = parseJWT(authHeader.slice(7));
```

```

    if (!token) {
        req.respond(403, {}, 'Invalid token\n');
        return;
    }
    req.ctx.userId = token.sub;
    req.ctx.roles = token.roles;
});

// Content handler: use claims from req.ctx
nginx.http.servers[0].locations[0].handler = function(req) {
    if (!req.ctx.roles.includes('admin')) {
        req.respond(403, {}, 'Forbidden\n');
        return;
    }
    req.respond(200, {}, 'Hello admin ' + req.ctx.userId + '\n');
};

```

10.2. Rate Limiting at the TCP Level

Accept hook with a per-IP token bucket stored in `nginx.shared`:

```

var srv = nginx.http.servers[0];

srv.on('accept', function(conn) {
    var key = 'rate:' + conn.remoteAddr;
    var now = Math.floor(Date.now() / 1000);
    var ts = parseInt(nginx.shared.get(key + ':ts') || '0');

    // Refill tokens (max 10, 1 per second)
    var tokens = parseInt(nginx.shared.get(key + ':t') || '10');
    if (now > ts) {
        tokens = Math.min(10, tokens + (now - ts));
        nginx.shared.set(key + ':ts', String(now));
    }

    if (tokens <= 0) {
        nginx.log(nginx.WARN, 'rate-limit: ' + conn.remoteAddr);
        conn.reject();
        return;
    }

    nginx.shared.set(key + ':t', String(tokens - 1));
});

```

10.3. Middleware Timing Wrapper

Wrap the entire hook chain for a location with request timing:

```

var loc = nginx.http.servers[0].locations.find(
    function(l) { return l.path === '/api/'; }
);

// Outer: measures total hook + handler time
loc.addHook(async function(req, next) {
    var t0 = Date.now();
    await next();
    nginx.log(nginx.INFO,
        req.method + ' ' + req.uri + ' ' + (Date.now() - t0) + 'ms');
});

// Inner: the actual auth check
loc.addHook(function(req) {
    if (!req.headers['x-api-key']) {
        req.respond(401, {}, 'Missing key\n');
    }
});

```

10.4. L4 Protocol Negotiation

Strip a custom binary handshake before HTTP parsing:

```

const MAGIC = 'NGXJS:';

nginx.http.servers[0].addL4Filter(async function*(source) {
    for await (const chunk of source) {
        var s = String.fromCharCode(...chunk);
        if (s.startsWith(MAGIC)) {
            nginx.log(nginx.INFO, 'ngxjs handshake from client');
            yield s.slice(MAGIC.length);
        } else {
            yield s; // plain HTTP – forward unchanged
        }
    }
    return; // one chunk is enough
});

```

10.5. Cross-Worker Hit Counter Plugin

A plugin loaded with `nginx.use()` that counts requests across all workers:

```

// /opt/plugins/hit-counter/index.js
var cfg = nginx.pluginConfig;
var location = cfg.location; // e.g. '/stats'

```

```

nginx.http.addHook(function(req) {
    nginx.shared.incr('hits');
});

var statsLoc = nginx.http.servers[0].locations.find(
    function(l) { return l.path === location; }
);
if (statsLoc) {
    statsLoc.handler = function(req) {
        var n = nginx.shared.get('hits') || '0';
        req.respond(200, {'Content-Type': 'text/plain'}, 'hits: ' + n + '\n');
    };
}

```

```

// js_source:
nginx.use('/opt/plugins/hit-counter', { location: '/stats' });

```

11. Reference

11.1. Hook API Summary

Method	Scope	When it fires
<code>nginx.http.addHook(fn)</code>	Global HTTP	Access phase — every request in the HTTP block
<code>server.addHook(fn)</code>	Virtual server	Access phase — every request on this server
<code>location.addHook(fn)</code>	Location	Content phase — every request matched to this location
<code>location.addResponseHook(fn)</code>	Location	After content handler selects headers, before they are sent

11.2. Accept Hook API Summary

Method	Target	Notes
<code>listener.on('accept', fn)</code>	JS-managed listener	JS-managed listeners created via <code>nginx.http.attach(sock)</code> .
<code>server.on('accept', fn)</code>	Standard listen socket	Sockets declared with nginx's <code>listen</code> directive.

11.3. L4 Filter API Summary

Method	Direction	Notes
<code>listener.addL4Filter(fn) / server.addL4Filter(fn)</code>	Client → nginx (inbound)	Before HTTP parsing. Async generator, <code>await</code> supported.
<code>listener.addL4SendFilter(fn) / server.addL4SendFilter(fn)</code>	Nginx → client (outbound)	After HTTP response is formed. Async generator, <code>await</code> supported.

11.4. Upstream Filter API Summary

Method	Direction
<code>location.addUpstreamFilter(fn)</code>	Upstream response body (server → client)
<code>location.addUpstreamRequestFilter(fn)</code>	Client request body (client → upstream)

11.5. `nginx.on` Event Reference

Event	Signature	Trigger
<code>workerSpawned</code>	<code>fn(pid, slot)</code>	Worker forked
<code>workerExited</code>	<code>fn(pid, slot, status)</code>	Worker exited
<code>quit</code>	<code>fn()</code>	SIGQUIT — graceful shutdown begins
<code>reload</code>	<code>fn()</code>	SIGHUP — config re-read begins
<code>reloaded</code>	<code>fn()</code>	New worker generation active
<code>reopen</code>	<code>fn()</code>	SIGUSR1 — log rotation

11.6. `nginx.shared` Limits

Limit	Value
Max key length	127 characters
Max value length	511 characters
Max entries	256
Zone size	≈ 163 kB

11.7. Error Handling Summary

Situation	Behaviour
Hook throws synchronously	nginx logs the error; 500 response sent.
Async hook's Promise rejects	nginx logs the error; 500 response sent.
Accept hook returns a Promise	Connection rejected; warning logged.
L4 filter generator throws	Connection closed; error logged.

Situation	Behaviour
<code>nginx.shared.set</code> when full	<code>InternalError</code> thrown to the caller.
<code>nginx.shared.set</code> with oversized key/value	<code>RangeError</code> thrown to the caller.
<code>nginx.on</code> callback throws	Error logged; remaining listeners for the same event still run.